COMP 4300
Intro to Game Programming

Lecture 20
Game Save & Load
Game Tools
Drag & Drop

Game Progression
Game Saving

Game Saving

- A vital part of any game is the ability to save the player's progress at some point
- Games can be saved in a number of ways
  - File Save, Quick Save, Checkpoint, etc
- Saved games can be stored in a number of ways as well, such as:
  - Local File, Cloud Save, Temp (RAM), Password

Saving Games

- When you allow players to save their progress is a game design decision
- Save/Load whenever they like can lead to much easier gameplay (quick save/load)
- Save after or mid-level (checkpoint)
- Saving only at specific locations (town / camp) can be inconvenient for the player

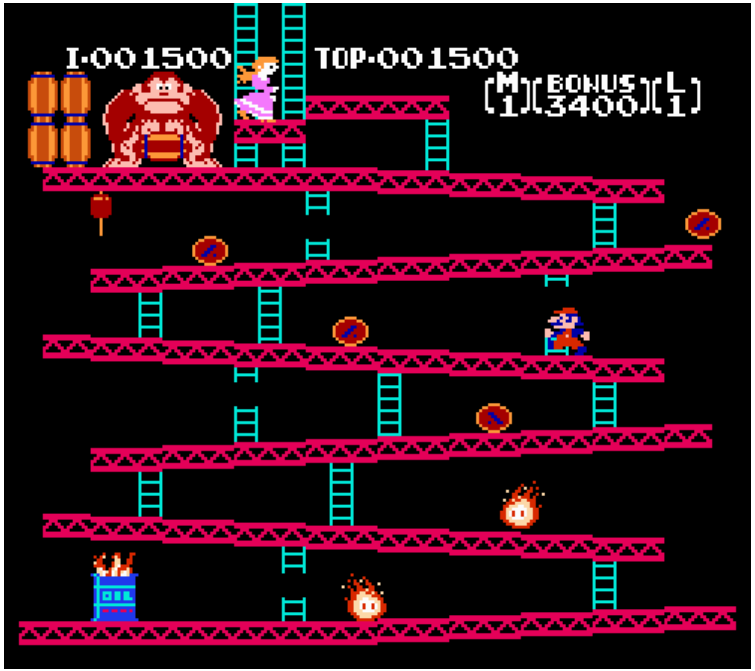
Saving Between Levels

- The easiest way to implement game save is between levels, when no entities are currently active in the game world
- In simple games (without quests, inventories, items, status etc) we can just save the level number that we are on

Displaying Progress

- Users progress must be conveyed somehow so that their progress can continue
- Simplest way: display level number (SMB)

Level Number Display



Displaying Progress

- Users progress must be conveyed somehow so that their progress can continue
- Simplest way: display level number (SMB)
- Another common way is to display progress as an ‘overworld map’ which conveys a deeper sense of immersion / progress


Necky's Nuts


EQUIP


EQUIP Y


Atlas
Highlight Maps
Type keywords here...
130 157
12:16
Menu
1
2
3
4
5
Shop
Q
W
E
R
T

Spyrian
level 15

Profile Summary
Hero Progress
Rewards
Daily Quests
Match History

Filter Heroes

Valla 268,098 XP to next level
Skin Variation

Sonya 180,690 XP to next level
GOLD: 500

Nazeebo 175,811 XP to next level
Advanced Talents

Raynor 30,000 XP to next level
Heroic Ability

Falstad 97,508 XP to next level
Heroic Ability

Displaying Progress

- Users progress must be conveyed somehow so that their progress can continue
- Simplest way: display level number (SMB)
- Another common way is to display progress as an ‘overworld map’ which conveys a deeper sense of immersion / progress
- RPG game may save at a location

Continue

- If you don’t want to (or can’t) implement a full save for your game
- Let the player continue from a given spot
- Restart Level? World? Etc
- Very common in early video games when arbitrary save game functionality may not have been possible

CONTINUE

SAVE

RETRY

CONTINUE 7

CONTINUE? 4

FREE PLAY


GAME OVER
CONTINUE
SAVE&CONTINUE
SAVE&QUIT

Save Points

- Save points make it easier to implement game saving / loading by limiting the places where players can save / load
- Typically static locations away from other entities and dangerous situations
- Allow us to load game without worrying about entity storage

Save Points


AUTO
ERGY 99
135
45
40

Checkpoints

- Level-based games may want a progress checkpoint placed within the level
- This reduces frustration by letting players save progress mid-level
- Doesn’t allow players to make game easier or reduce RNG like quick save

Checkpoints

Checkpoint Implementation

- Checkpoints are easy to implement
- If level has a finite number of checkpoints, just store the checkpoint number along with each level
- When player enters a level, spawn them at the position of the checkpoint stored
- Reset checkpoints when level completed

AutoSave

- Most modern games use some sort of automatic saving functionality
- Nearly identical to Checkpoint saving, but happens invisibly behind the scenes
- Gameplay decision on when to save
  - Player enters new ”zone”
  - Player kills a boss
  - Player picks up an item, etc

This game saves data automatically. Do not turn off
the game when the saving icon is visible.

What Data to Save?

- Depends on saving functionality
- Save After Level, Store:
  - Current levels completed, inventory, weapons
- Save at Save Point:
  - Quests completed, party stats, save point used
- Quick Save:
  - Entity game state, each entity’s components

Loading Saved Game

- How to implement loading a saved game depends on what data was saved
- If level / items stored, just recreate the game before that level starts
- If quicksave, may have to load all entities back so that game resumes from QS point

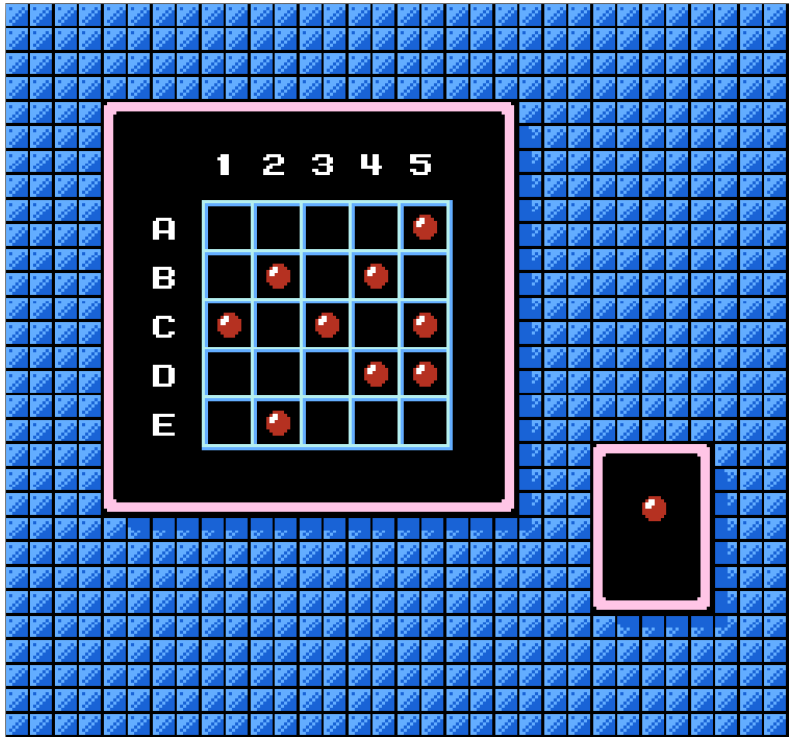
Loading Ex: Overworld View

- Saved game stores levels completed, current Mario status / inventory (SMB3)
- Game is loaded, Mario appears on the overworld map with completed maps shown and inventory available
- Game load may require quit to main menu

Loading Ex: Quick Save

- In many FPS games, press a button at any time to instantly save
- When quickload happens, all entities and game progress resumes from when the button was pressed
- Players can abuse this to get past difficult areas / know the outcome of RNG etc

Password Save



Bubble Bobble passwords

Artificial password synthesis

```
LevelNumber =  1 = 0000001 → aaabbbc
SuperFlag   =  1 =       1 → f
Something   =  3 =      11 → mm

Letter0 : 0000                = 0000 = 0 = B
Letter1 : 0000 ⊕ Letter0’ = 0000 = 0 = B
Letter2 : 0011 ⊕ Letter1’ = 0011 = 3 = F
Letter3 : 0111 ⊕ Letter2’ = 0100 = 4 = J
Letter4 : 0000 ⊕ Letter3’ = 0100 = 4 = J

CheckSum    =  8 =  01000  → ssuuu
```

NES Punchout Password

https://www.youtube.com/watch?v=ap1YL_kGGlg

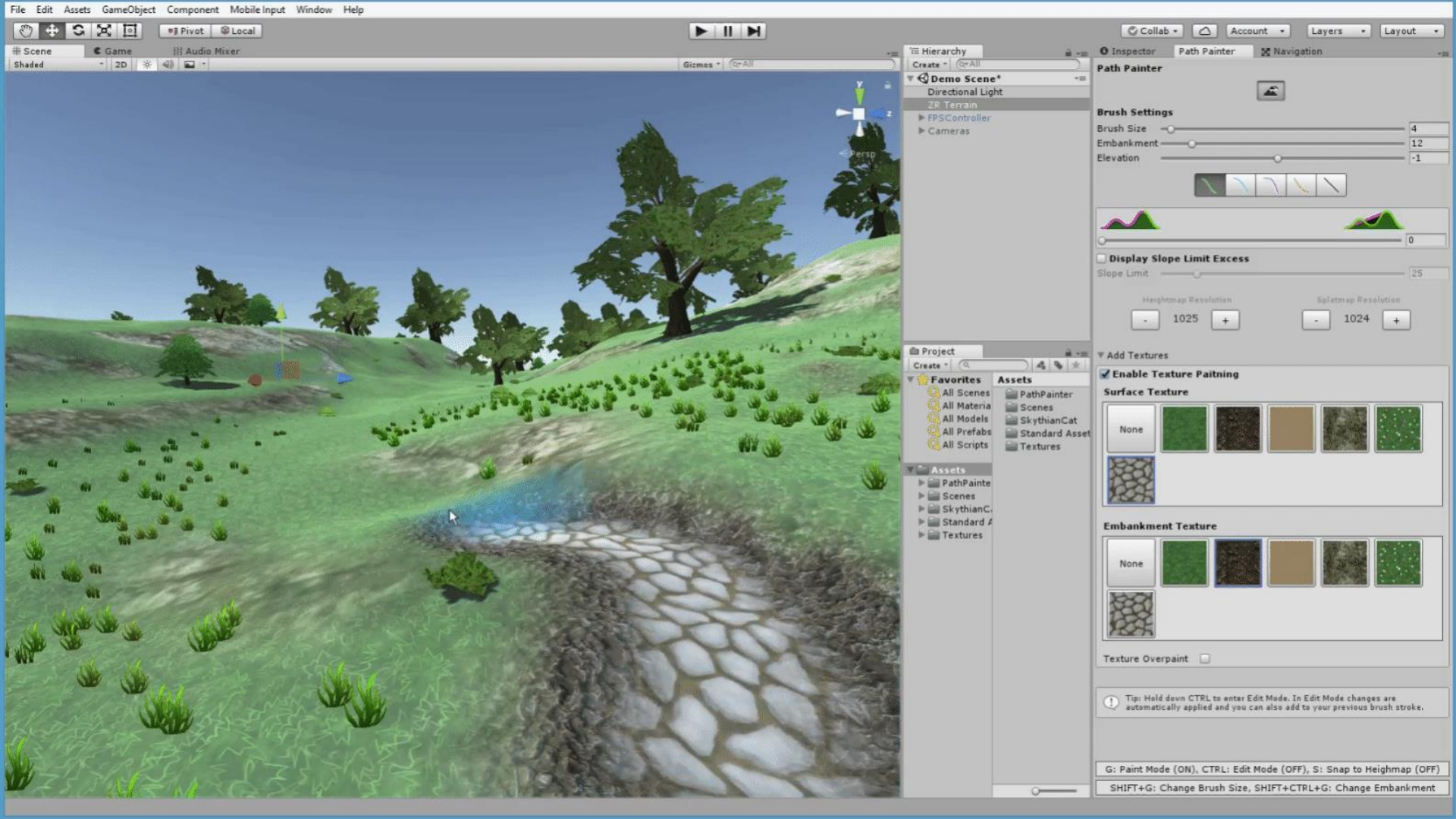
Game Tools

Game Tools

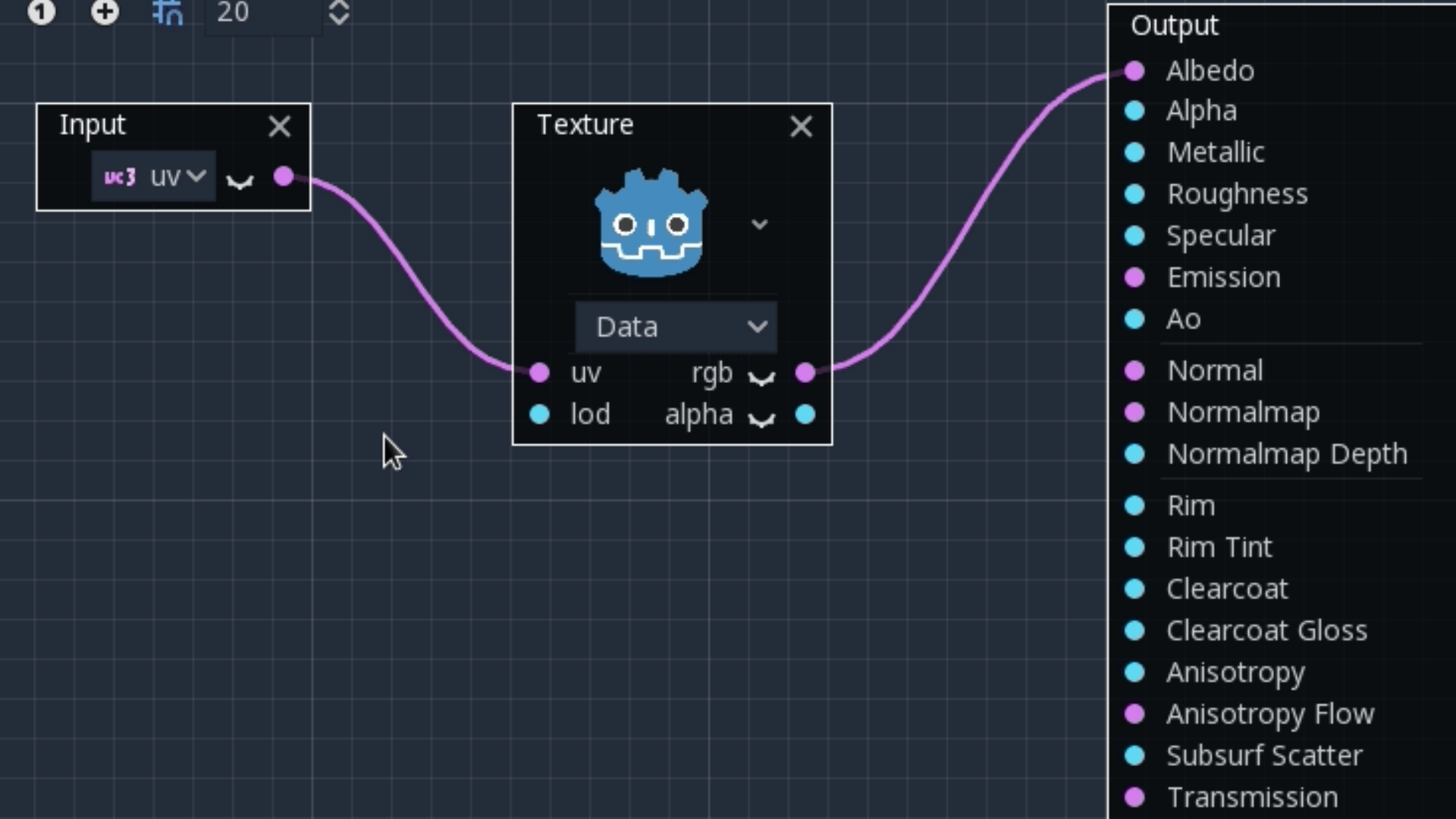
- As you make more games, you quickly learn that content creation becomes one of the most time consuming aspects of video game development
- Tools are programs that assist us with creating content for our games
- Many tools exist to help with general asset creation
  - Photoshop, Blender, PixelArt, Notepad, etc
- For more specific asset creation (maps, levels) we need to construct our own tools to help us

Popular Game Engines

- Several popular game engines exist that come with tools for creating content
  - Unity, Unreal, GameMaker, etc
- These game engines are more general, also help with game programming etc
- They do a great job of illustrating the power of tools for game creation


File Edit Assets GameObject Component Mobile Input Window Help
Pivot Local
Collab Account Layers Layout
Scene Game Audio Mixer
Shaded 2D Gizmos
Persp
Hierarchy
Create
Demo Scene*
Directional Light
ZR Terrain
FPSController
Cameras
Project
Create
Favorites
All Scenes
All Materia
All Models
All Prefabs
All Scripts
Assets
PathPainter
Scenes
SkythianCat
Standard Asset
Textures
Assets
PathPainte
Scenes
SkythianC
Standard A
Textures
Inspector
Path Painter
Navigation
Path Painter
Brush Settings
Brush Size 4
Embankment 12
Elevation -1
0
Display Slope Limit Excess
Slope Limit 25
Heightmap Resolution
1025
Splatmap Resolution
1024
Add Textures
Enable Texture Paitning
Surface Texture
None
Embankment Texture
None
Texture Overpaint
Tip: Hold down CTRL to enter Edit Mode. In Edit Mode changes are automatically applied and you can also add to your previous brush stroke.
G: Paint Mode (ON), CTRL: Edit Mode (OFF), S: Snap to Heighmap (OFF)
SHIFT+G: Change Brush Size, SHIFT+CTRL+G: Change Embankment

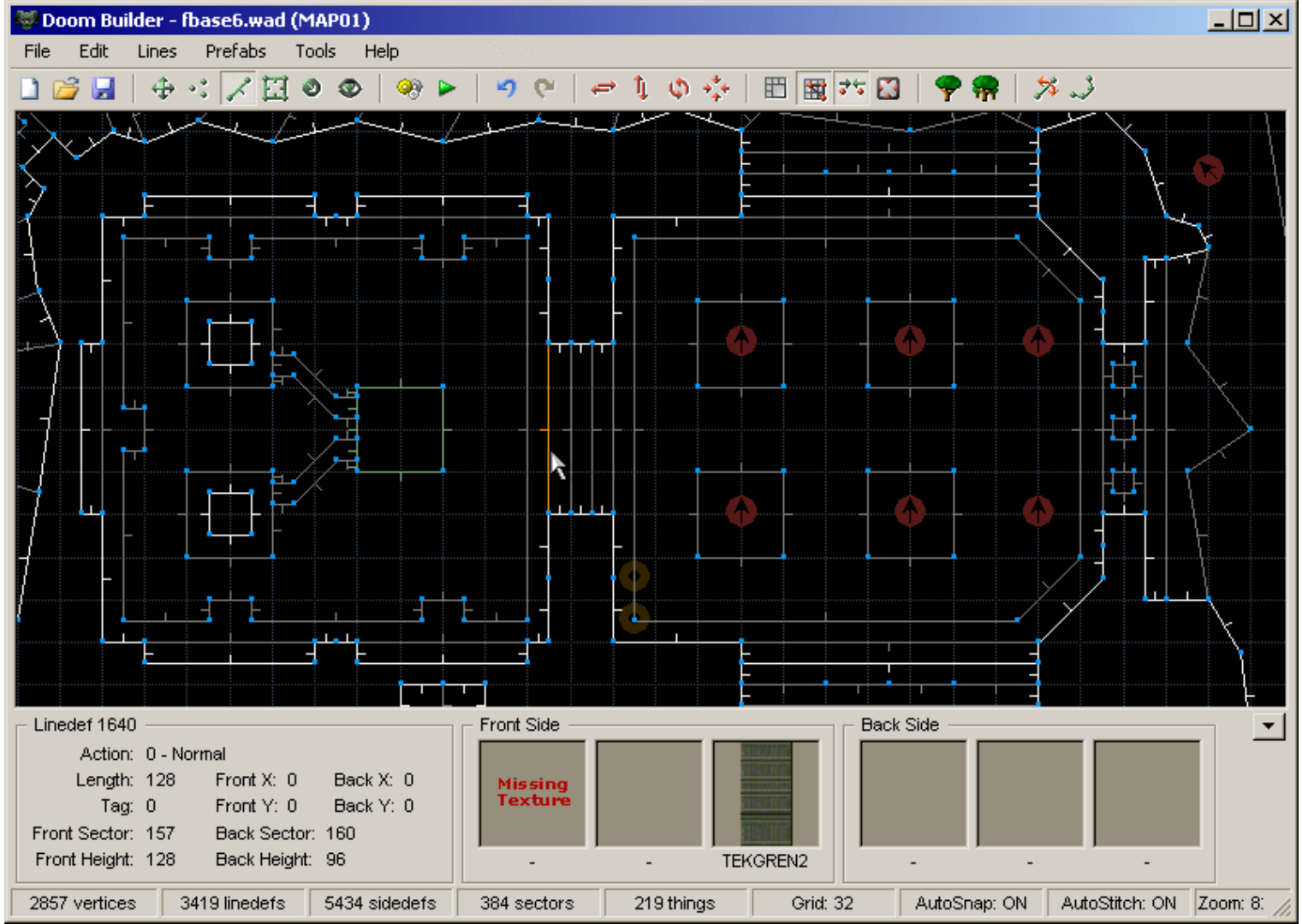

Persp

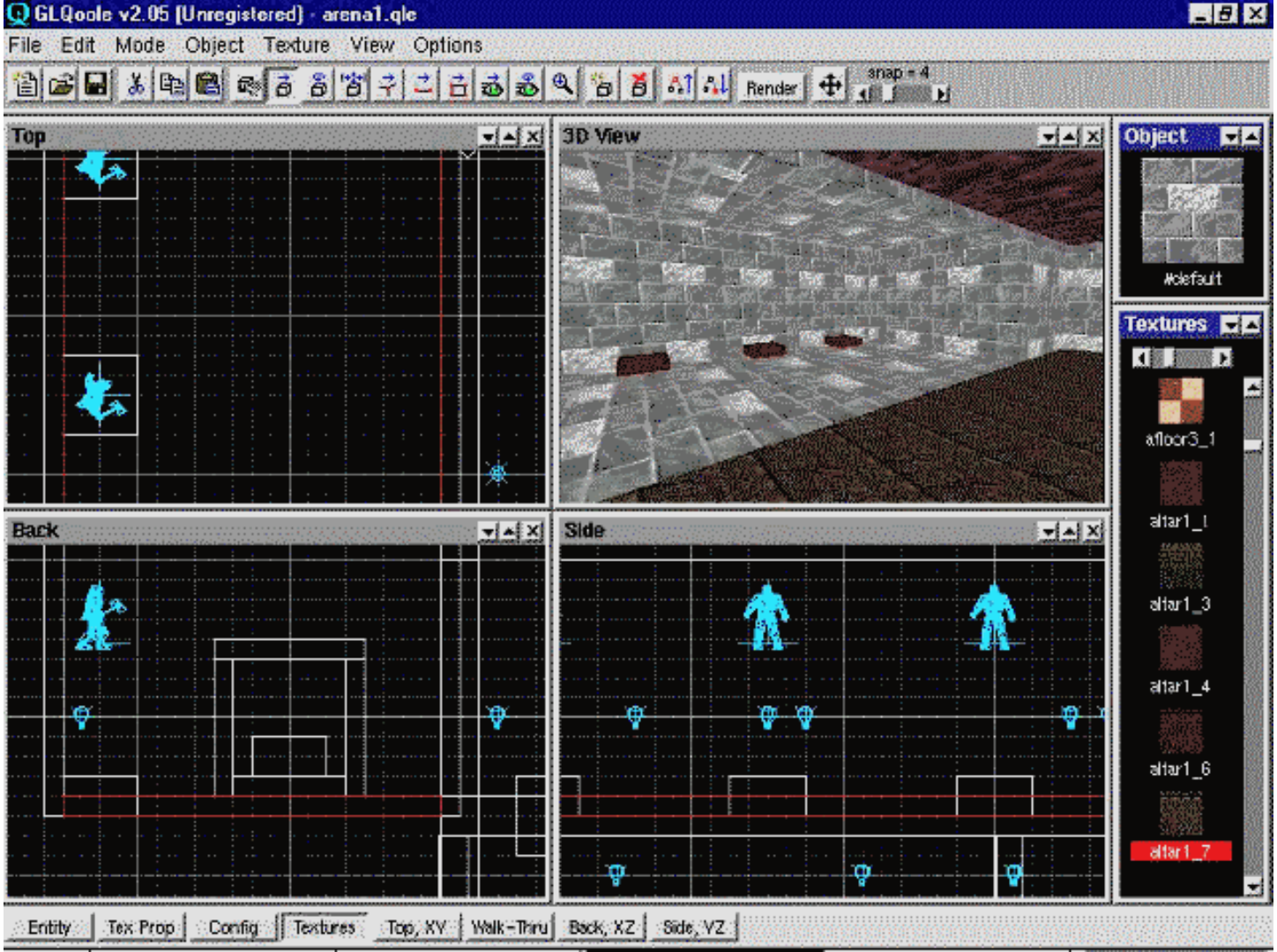

20
Input
uv
uv
Texture
Data
uv
rgb
lod
alpha
Output
Albedo
Alpha
Metallic
Roughness
Specular
Emission
Ao
Normal
Normalmap
Normalmap Depth
Rim
Rim Tint
Clearcoat
Clearcoat Gloss
Anisotropy
Anisotropy Flow
Subsurf Scatter
Transmission

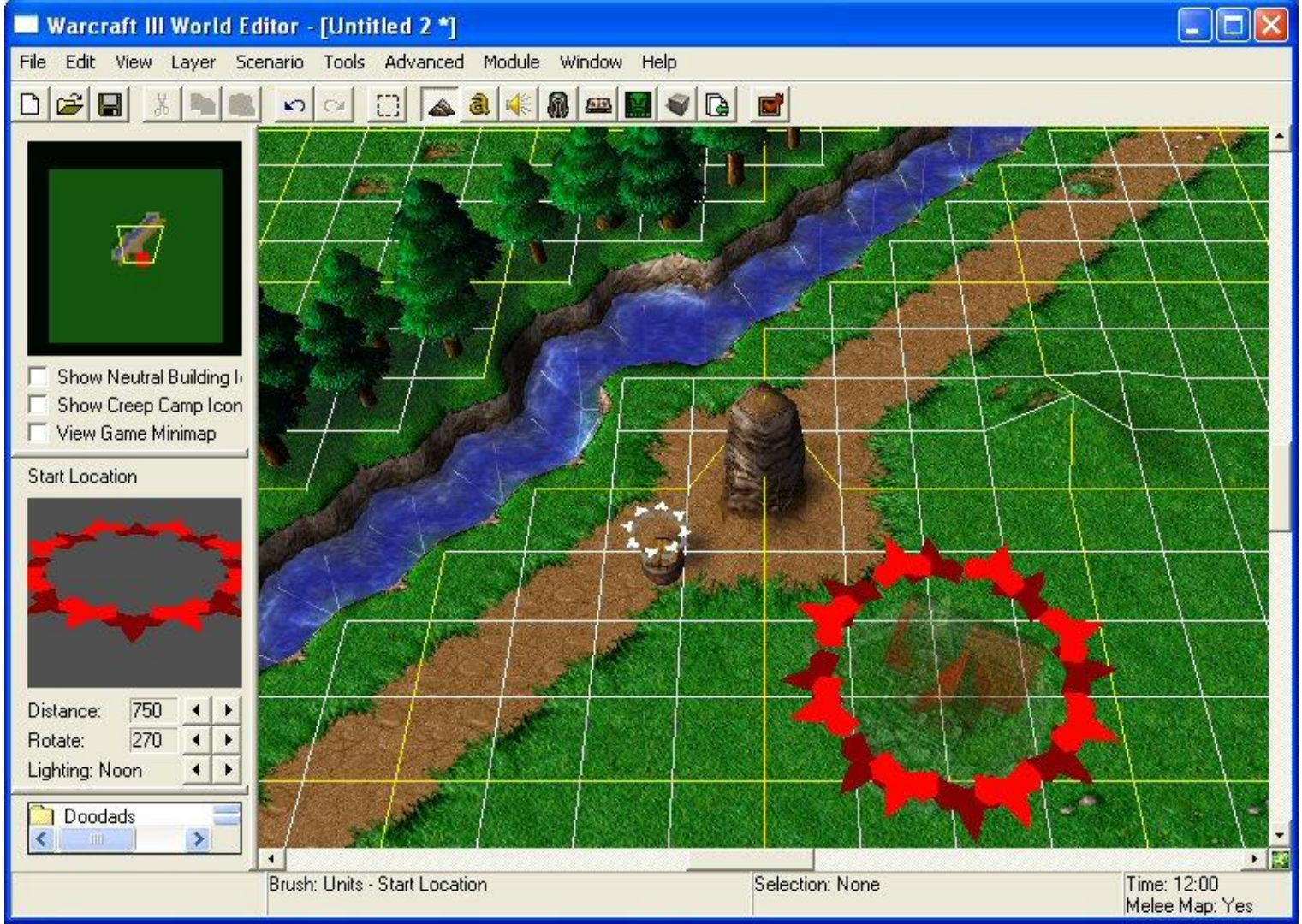
Custom Game Tools

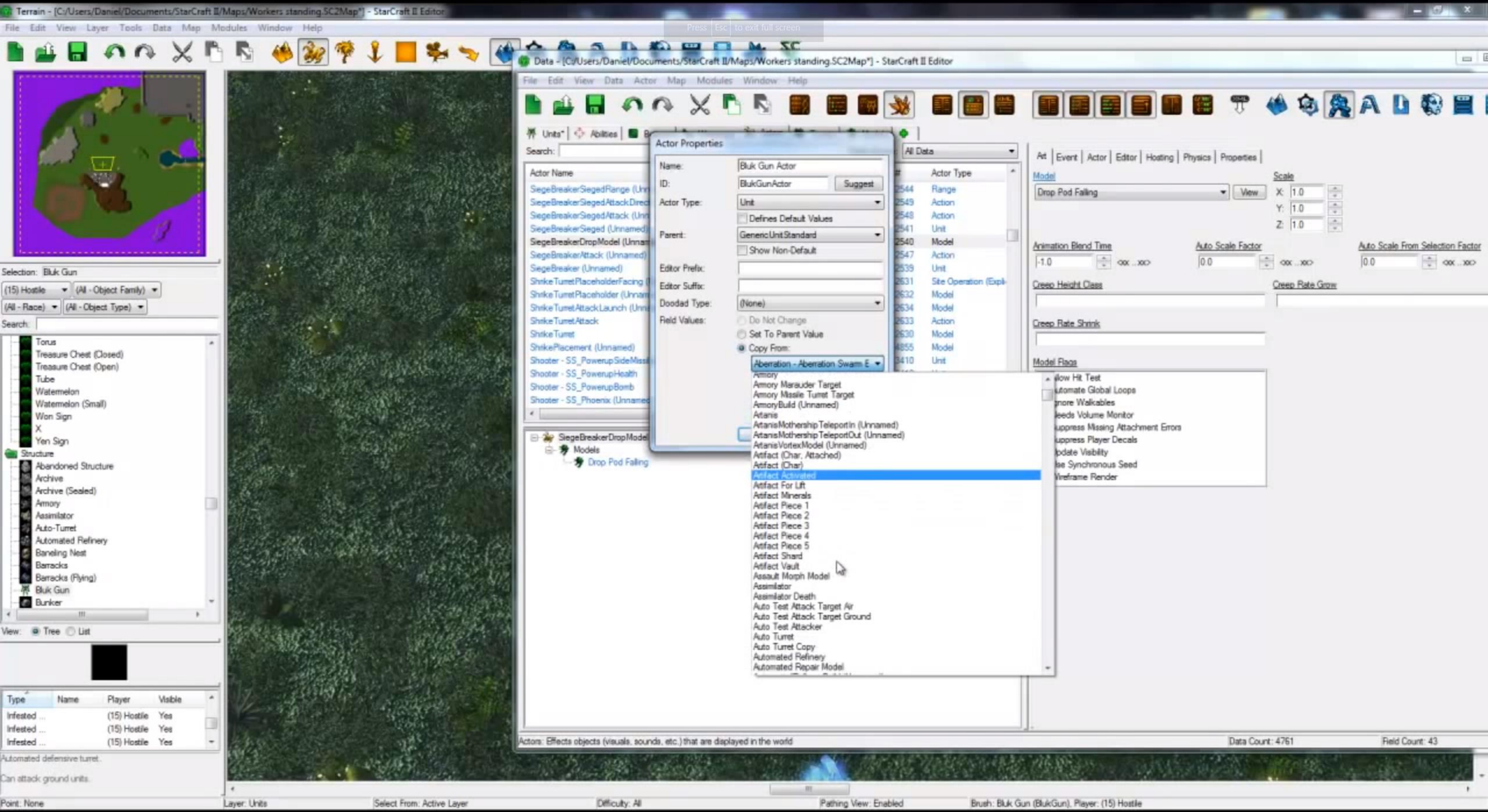
- If your game is not using a pre-existing engine, you must create your own tools
- Many existing games (older games) come with editors/tools for modding or creating other custom content
- The features and power of these tools vary wildly from game to game


Warcraft 2 Editor - [Unknown Level Orcs.vv4]
File Edit View Tools Owner Window Help
Mini...
About WarCraft 2 Editor
WarCraft 2 Editor Version 0.4 Alpha
Blizzard Entertainment, Copyright © 1995
OK
For Help, press F1
27, 12


Doom Builder - fbase6.wad (MAP01)
File
Edit
Lines
Prefabs
Tools
Help
Linedef 1640
Action: 0 - Normal
Length: 128
Front X: 0
Back X: 0
Tag: 0
Front Y: 0
Back Y: 0
Front Sector: 157
Back Sector: 160
Front Height: 128
Back Height: 96
Front Side
Missing Texture
-
-
TEKGREN2
Back Side
-
-
-
2857 vertices
3419 linedefs
5434 sidedefs
384 sectors
219 things
Grid: 32
AutoSnap: ON
AutoStitch: ON
Zoom: 8:


GLQoole v2.05 [Unregistered] - arena1.qle
File
Edit
Mode
Object
Texture
View
Options
Render
snap = 4
Top
3D View
Object
#default
Textures
afloor3_1
altar1_1
altar1_3
altar1_4
altar1_6
altar1_7
Back
Side
Entity
Tex Prop
Config
Textures
Top, XY
Walk-Thru
Back, XZ
Side, YZ


Warcraft III World Editor - [Untitled 2 *]
File Edit View Layer Scenario Tools Advanced Module Window Help
Show Neutral Building I
Show Creep Camp Icon
View Game Minimap
Start Location
Distance: 750
Rotate: 270
Lighting: Noon
Doodads
Brush: Units - Start Location
Selection: None
Time: 12:00
Melee Map: Yes


Terrain - [C:/Users/Daniel/Documents/StarCraft II/Maps/Workers standing.SC2Map*] - StarCraft II Editor
File Edit View Layer Tools Data Map Modules Window Help
Data - [C:/Users/Daniel/Documents/StarCraft II/Maps/Workers standing.SC2Map*] - StarCraft II Editor
File Edit View Data Actor Map Modules Window Help
Actor Properties
Name: Bluk Gun Actor
ID: BlukGunActor
Suggest
Actor Type: Unit
Defines Default Values
Parent: GenericUnitStandard
Show Non-Default
Editor Prefix:
Editor Suffix:
Doodad Type: (None)
Field Values:
Do Not Change
Set To Parent Value
Copy From:
Aberration - Aberration Swarm E
Armory Marauder Target
Armory Missile Turret Target
ArmoryBuild (Unnamed)
Artanis
ArtanisMothershipTeleportIn (Unnamed)
ArtanisMothershipTeleportOut (Unnamed)
ArtanisVortexModel (Unnamed)
Artifact (Char, Attached)
Artifact (Char)
Artifact Activated
Artifact For Lift
Artifact Minerals
Artifact Piece 1
Artifact Piece 2
Artifact Piece 3
Artifact Piece 4
Artifact Piece 5
Artifact Shard
Artifact Vault
Assault Morph Model
Assimilator
Assimilator Death
Auto Test Attack Target Air
Auto Test Attack Target Ground
Auto Test Attacker
Auto Turret
Auto Turret Copy
Automated Refinery
Automated Repair Model
Actor Name
Actor Type
SiegeBreakerSiegedRange
SiegeBreakerSiegedAttackDirect
SiegeBreakerSiegedAttack
SiegeBreakerSieged (Unnamed)
SiegeBreakerDropModel
SiegeBreakerAttack (Unnamed)
SiegeBreaker (Unnamed)
ShrikeTurretPlaceholderFacing
ShrikeTurretPlaceholder
ShrikeTurretAttackLaunch
ShrikeTurretAttack
ShrikeTurret
ShrikePlacement (Unnamed)
Shooter - SS_PowerupSideMissi
Shooter - SS_PowerupHealth
Shooter - SS_PowerupBomb
Shooter - SS_Phoenix (Unname
Range
Action
Action
Unit
Model
Action
Unit
Site Operation (Expl
Model
Model
Action
Model
Model
Unit
SiegeBreakerDropModel
Models
Drop Pod Falling
Art Event Actor Editor Hosting Physics Properties
Model
Drop Pod Falling
View
Scale
X: 1.0
Y: 1.0
Z: 1.0
Animation Blend Time
Auto Scale Factor
Auto Scale From Selection Factor
Creep Height Class
Creep Rate Grow
Creep Rate Shrink
Model Flags
Automate Global Loops
Needs Volume Monitor
Update Visibility
Wireframe Render
Actors: Effects objects (visuals, sounds, etc.) that are displayed in the world
Data Count: 4761
Field Count: 43
Selection: Bluk Gun
(15) Hostile
(All - Object Family)
(All - Race)
(All - Object Type)
Search:
Torus
Treasure Chest (Closed)
Treasure Chest (Open)
Tube
Watermelon
Watermelon (Small)
Won Sign
X
Yen Sign
Structure
Abandoned Structure
Archive
Archive (Sealed)
Armory
Assimilator
Auto-Turret
Automated Refinery
Baneling Nest
Barracks
Barracks (Flying)
Bluk Gun
Bunker
View: Tree List
Type Name Player Visible
Infested (15) Hostile Yes
Infested (15) Hostile Yes
Infested (15) Hostile Yes
Automated defensive turret.
Can attack ground units.
Point: None
Layer: Units
Select From: Active Layer
Difficulty: All
Pathing View: Enabled
Brush: Bluk Gun (BlukGun), Player: (15) Hostile


SUPER MARIO BROS.
PLAY


gifs.com

Tools Implementation

- Game tools are typically made along with a graphical user interface (GUI), which includes some sort of menu system
- The first step will be drag-and-drop

Drag and Drop

Drag and Drop

- Drag and drop typically refers to the ability to move an on-screen UI element from one place to another with the mouse
- In order to implement a basic level editing tool for our games, we can implement drag-and-drop via ECS
- Let’s create a Draggable component

Drag and Drop Variant

- Literal drag and drop would implement:
  1. Mouse Down on entity = start drag
  2. As mouse moves, move entity
  3. Mouse Release = let go
- Kind of annoying in practice, instead let's implement 'pick up and put down'
  1. Mouse down on entity = pick up
  2. As mouse moves, move entity
  3. Click to place final location and let go

Draggable Component

```
1. class CDraggable
2. {
3. public:
4.     bool dragging = false;
5.     CDraggable() {}
6. }
```

Draggable Event

```
1.  void sInput()
2.  {
3.      if (left mouse button pressed)
4.      {
5.          auto e = closestDraggableEntityToMouseClickPosition();
6.          auto & d = e->getComponent<CDraggable>().dragging;
7.          // if we are currently dragging, let go of the entity
8.          if (d) { d = false; }
9.          // otherwise if we clicked inside the entity, start dragging
10.         else if (mouse click is inside e's animation) { d = true; }
11.     }
12. }
```

Draggable System

```
1.  void sDragAndDrop()
2.  {
3.      for (auto e : entities)
4.      {
5.          if (e->hasComponent<CDraggable>() &&
6.              e->getComponent<CDraggable>.dragging)
7.          {
8.              // set e position to the current mouse position
9.          }
10.     }
11. }
```